Laboratory reportof Nanjing University of Information Science and TechnologyExperiment

Title　Lab 8　Date　2025.6.6　Class　24AC

Name　ZhuangZhong　ID　202483910032

1. Experimental Purpose

Practice Version Control: Learn to use Docker Tags to manage different iterations of a cloud application.

Validate Decoupled Deployment: Simulate a disaster recovery scenario to verify the ability to deploy services in a remote environment without source code.

Security Best Practices: Implement Secret Management using Personal Access Tokens (PAT) instead of plaintext passwords.

2. Experiment Contents

This lab completes the transition from a "Standalone Developer" to a "Cloud Engineer":

Configure secure authentication with GitHub Container Registry (GHCR).

Tag and push your web application image to the cloud.

Perform a version upgrade (v1.0 → v2.0) and observe the Layered Storage mechanism.

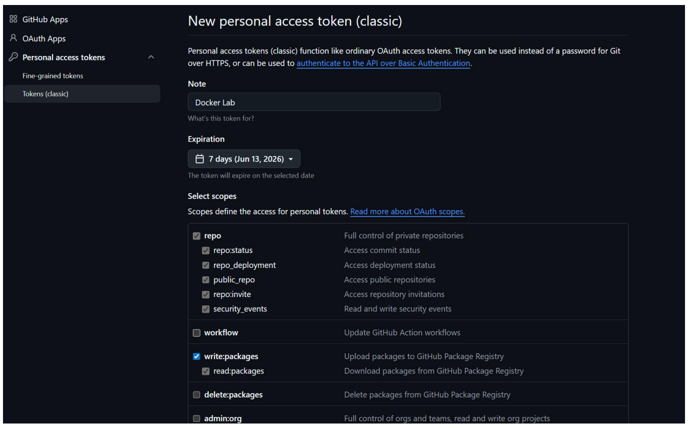
Perform a "Clean-room Deployment" by pulling your image onto a fresh environment after wiping local data.

3. Detailed Steps of the Experiment

Step 1: Secure Authentication - Generate Personal Access Token (PAT)

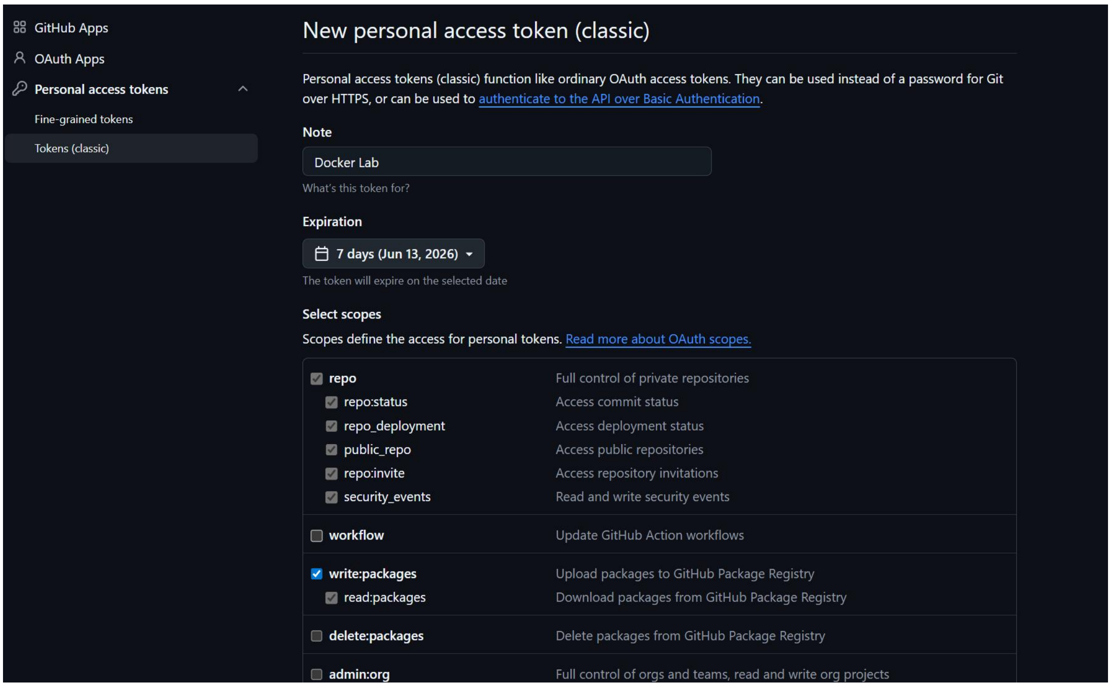
1.1 Navigate to Token Creation Page

GitHub Settings → Developer settings → Personal access tokens → Tokens (classic)



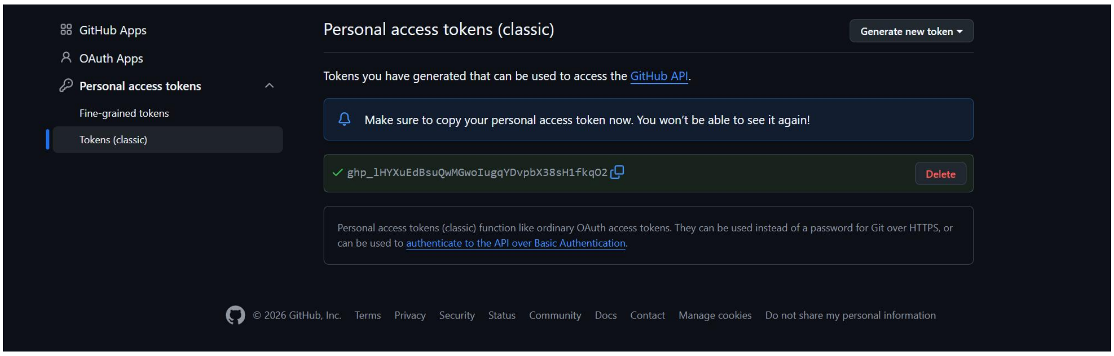
1.2 Configure Permissions

Check write:packages (automatically includes repo and read:packages)

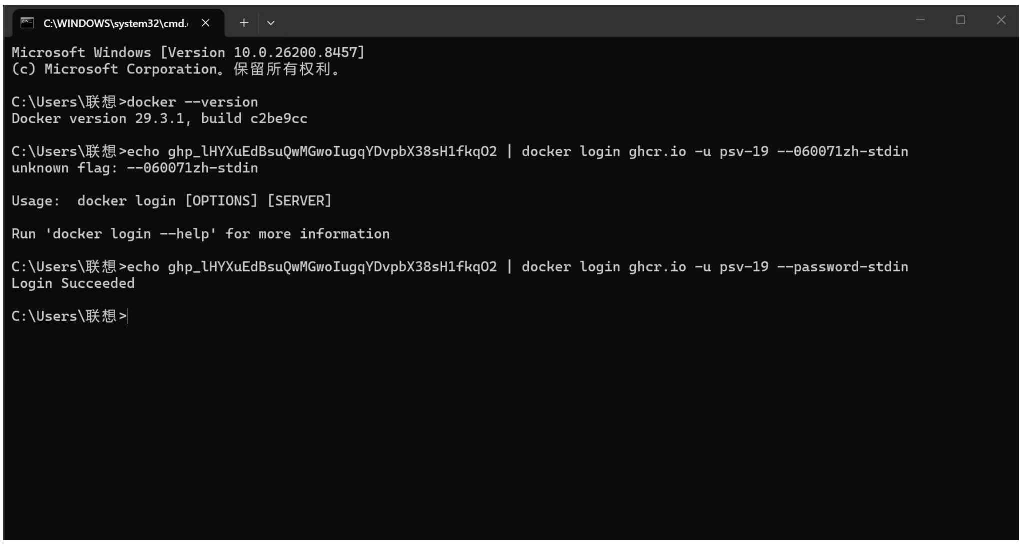


1.3 Save Token

Click Generate token and copy immediately



1.4 Login via Terminal



Step 2: Prepare Project Files
cmd
cd %USERPROFILE%
mkdir docker-lab
cd docker-lab
echo ^<h1^>Version 1.0^</h1^> > index.html
echo FROM nginx:alpine > Dockerfile
echo COPY index.html /usr/share/nginx/html/ >> Dockerfile

```
Login Succeeded

C:\Users\联想>cd %USERPROFILE%

C:\Users\联想>mkdir docker-lab

C:\Users\联想>cd docker-lab

C:\Users\联想\docker-lab>echo ^<h1^>Version 1.0^</h1^> > index.html

C:\Users\联想\docker-lab>echo FROM nginx:alpine > Dockerfile

C:\Users\联想\docker-lab>echo COPY index.html /usr/share/nginx/html/ >> Docker

C:\Users\联想\docker-lab>dir
 Volume in drive C has no label.
 Volume Serial Number is C08C-08C7

 Directory of C:\Users\联想\docker-lab

2026/06/06  15:09    <DIR>          .
2026/06/06  15:09    <DIR>          ..
2026/06/06  15:09                61 Dockerfile
2026/06/06  15:09                23 index.html
               2 File(s)             84 bytes
               2 Dir(s)  40,500,203,520 bytes free
```

Step 3: Configure Image Accelerator (To solve network issues)
Due to the local network environment, Docker image accelerators were configured:

```
  com.docker.desktop.address=npipe://\\.\pipe\docker_cli
 Experimental: false
 Insecure Registries:
  hubproxy.docker.internal:5555
  ::1/128
  127.0.0.0/8
 Registry Mirrors:
  https://docker.1ms.run/
  https://docker.xuanyuan.me/
  https://atomhub.openatom.cn/
 Live Restore Enabled: false
 Firewall Backend: iptables


C:\Users\联想\docker-lab>docker pull nginx:alpine
alpine: Pulling from library/nginx
fbaed3f7fcbe: Pulling fs layer
a5008f4a4b25: Download complete
abaae85d1626: Pulling fs layer
de1b677d8c00: Download complete
43f834d60d8a: Download complete
6a0ac1617861: Pulling fs layer
e654dbbbb9e1: Pulling fs layer
94d083cf706a: Download complete
ab556c9e4280: Download complete
```

Step 4: Build and Push v1.0 Image
4.1 Build the image

cmd

docker build -t cloud-lab-image:v1 .

4.2 Check images

cmd

docker images | findstr cloud-lab

```
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/k1jv9n6qyinz9q1fihia1liqz

C:\Users\联想\docker-lab>docker images | findstr cloud-lab
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
cloud-lab-image:v1                                                                   fa485401797d       92.7MB          26MB

C:\Users\联想\docker-lab>echo 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 | docker login ghcr.io -u psv-19 --password-stdin
Login Succeeded

C:\Users\联想\docker-lab>docker tag cloud-lab-image:v1 ghcr.io/psv-19/cloud-app:v1.0

C:\Users\联想\docker-lab>docker push ghcr.io/psv-19/cloud-app:v1.0
The push refers to repository [ghcr.io/psv-19/cloud-app]
43f834d60d8a: Pushed
e654dbbbb9e1: Pushed
94d083cf706a: Pushed
6a0ac1617861: Pushed
a5008f4a4b25: Pushed
fbaed3f7fcbe: Pushed
bd5df978c10b: Pushed
90d05db799c7: Pushed
de1b677d8c00: Pushed
abaae85d1626: Pushed
v1.0: digest: sha256:fa485401797d3f326a09de828a8fa0fdddf491b382ad8bc248b2e7607ced97da size: 856
```

4.3 Login to GHCR (confirmation)

cmd

echo ghp_xxx | docker login ghcr.io -u psv-19 --password-stdin


```
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/k1jv9n6qyinz9q1fihia1liqz

C:\Users\联想\docker-lab>docker images | findstr cloud-lab
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
cloud-lab-image:v1                                                                   fa485401797d       92.7MB          26MB

C:\Users\联想\docker-lab>echo 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 | docker login ghcr.io -u psv-19 --password-stdin
Login Succeeded

C:\Users\联想\docker-lab>docker tag cloud-lab-image:v1 ghcr.io/psv-19/cloud-app:v1.0

C:\Users\联想\docker-lab>docker push ghcr.io/psv-19/cloud-app:v1.0
The push refers to repository [ghcr.io/psv-19/cloud-app]
43f834d60d8a: Pushed
e654dbbbb9e1: Pushed
94d083cf706a: Pushed
6a0ac1617861: Pushed
a5008f4a4b25: Pushed
fbaed3f7fcbe: Pushed
bd5df978c10b: Pushed
90d05db799c7: Pushed
de1b677d8c00: Pushed
abaae85d1626: Pushed
v1.0: digest: sha256:fa485401797d3f326a09de828a8fa0fdddf491b382ad8bc248b2e7607ced97da size: 856
```


4.4 Tag and Push

cmd

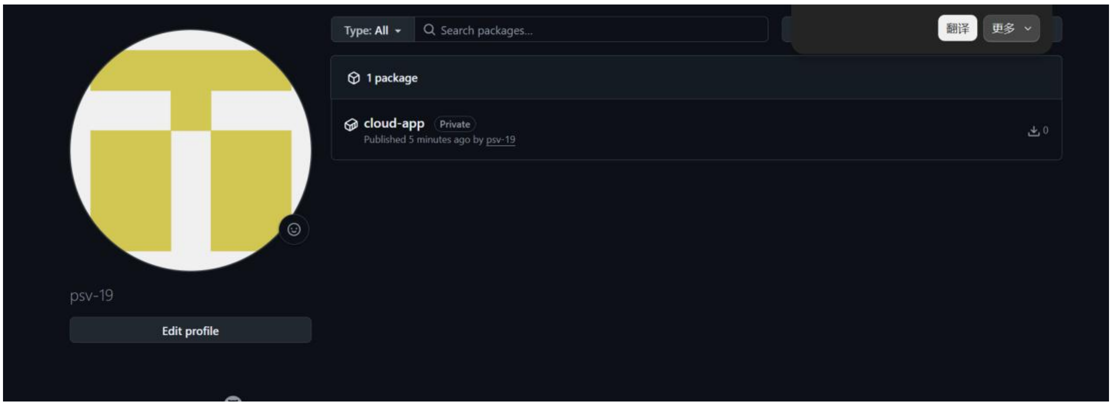
docker tag cloud-lab-image:v1 ghcr.io/psv-19/cloud-app:v1.0

docker push ghcr.io/psv-19/cloud-app:v1.0


```
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/k1jv9n6qyinz9q1fihia1liqz

C:\Users\联想\docker-lab>docker images | findstr cloud-lab
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
cloud-lab-image:v1                                                                   fa485401797d       92.7MB          26MB

C:\Users\联想\docker-lab>echo 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 | docker login ghcr.io -u psv-19 --password-stdin
Login Succeeded

C:\Users\联想\docker-lab>docker tag cloud-lab-image:v1 ghcr.io/psv-19/cloud-app:v1.0

C:\Users\联想\docker-lab>docker push ghcr.io/psv-19/cloud-app:v1.0
The push refers to repository [ghcr.io/psv-19/cloud-app]
43f834d60d8a: Pushed
e654dbbbb9e1: Pushed
94d083cf706a: Pushed
6a0ac1617861: Pushed
a5008f4a4b25: Pushed
fbaed3f7fcbe: Pushed
bd5df978c10b: Pushed
90d05db799c7: Pushed
de1b677d8c00: Pushed
abaae85d1626: Pushed
v1.0: digest: sha256:fa485401797d3f326a09de828a8fa0fdddf491b382ad8bc248b2e7607ced97da size: 856
```


4.5 Verify on GitHub Packages

Visit GitHub Profile → Packages



Step 5: Version Iteration v2.0 & Layer Analysis

5.1 Modify code

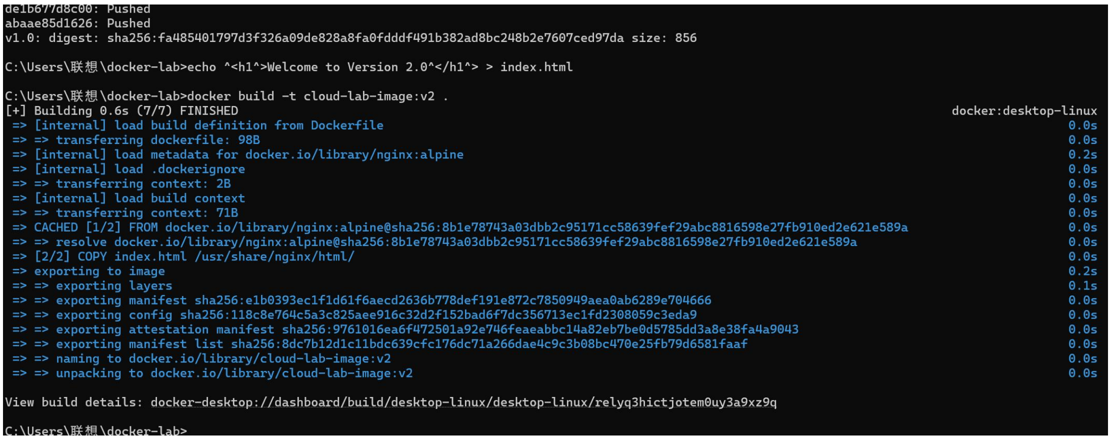
cmd

echo ^<h1^>Welcome to Version 2.0^</h1^> > index.html

5.2 Rebuild

cmd

docker build -t cloud-lab-image:v2 .



```
de1b677d8c00: Pushed
abaae85d1626: Pushed
v1.0: digest: sha256:fa485401797d3f326a09de828a8fa0fdddf491b382ad8bc248b2e7607ced97da size: 856

C:\Users\联想\docker-lab>echo ^<h1^>Welcome to Version 2.0^</h1^> > index.html

C:\Users\联想\docker-lab>docker build -t cloud-lab-image:v2 .
[+] Building 0.6s (7/7) FINISHED                                                docker:desktop-linux
 => [internal] load build definition from Dockerfile                                            0.0s
 => => transferring dockerfile: 98B                                                             0.0s
 => [internal] load metadata for docker.io/library/nginx:alpine                                 0.2s
 => [internal] load .dockerignore                                                               0.0s
 => => transferring context: 2B                                                                 0.0s
 => [internal] load build context                                                               0.0s
 => => transferring context: 71B                                                                0.0s
 => CACHED [1/2] FROM docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a  0.0s
 => => resolve docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a  0.0s
 => [2/2] COPY index.html /usr/share/nginx/html/                                                0.0s
 => exporting to image                                                                          0.2s
 => => exporting layers                                                                         0.1s
 => => exporting manifest sha256:e1b0393ec1f1d61f6aecd2636b778def191e872c7850949aea0ab6289e704666  0.0s
 => => exporting config sha256:118c8e764c5a3c825aee916c32d2f152bad6f7dc356713ec1fd2308059c3eda9  0.0s
 => => exporting attestation manifest sha256:9761016ea6f472501a92e746feaeabbc14a82eb7be0d5785dd3a8e38fa4a9043  0.0s
 => => exporting manifest list sha256:8dc7b12d1c11bdc639cfc176dc71a266dae4c9c3b08bc470e25fb79d6581faaf  0.0s
 => => naming to docker.io/library/cloud-lab-image:v2                                           0.0s
 => => unpacking to docker.io/library/cloud-lab-image:v2                                        0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/relyq3hictjotem0uy3a9xz9q

C:\Users\联想\docker-lab>
```

5.3 Push v2.0

cmd

docker tag cloud-lab-image:v2 ghcr.io/psv-19/cloud-app:v2.0

docker push ghcr.io/psv-19/cloud-app:v2.0

Key Observation: The output shows many Layer already exists messages; only the modified HTML layer is newly pushed.

Step 6: Disaster Recovery Test

6.1 Wipe all local data

cmd

docker system prune -a

Type y to confirm

```
Total reclaimed space: 818.1MB

C:\Users\联想\docker-lab>docker run -d -p 8888:80 ghcr.io/psv-19/cloud-app:v1.0
Unable to find image 'ghcr.io/psv-19/cloud-app:v1.0' locally
v1.0: Pulling from psv-19/cloud-app
90d05db799c7: Pull complete
e654dbbbb9e1: Pull complete
abaae85d1626: Pull complete
94d083cf706a: Pull complete
fbaed3f7fcbe: Pull complete
43f834d60d8a: Pull complete
a5008f4a4b25: Pull complete
de1b677d8c00: Pull complete
6a0ac1617861: Pull complete
bd5df978c10b: Download complete
Digest: sha256:fa485401797d3f326a09de828a8fa0fdddf491b382ad8bc248b2e7607ced97da
Status: Downloaded newer image for ghcr.io/psv-19/cloud-app:v1.0
675475e96004b23231c91d2f759ceafd0fc2551ba46bc106e04043c869948cf9

C:\Users\联想\docker-lab>
```

6.2 Pull and run v1.0 from cloud
cmd
docker run -d -p 8888:80 ghcr.io/psv-19/cloud-app:v1.0


```
Total reclaimed space: 818.1MB

C:\Users\联想\docker-lab>docker run -d -p 8888:80 ghcr.io/psv-19/cloud-app:v1.0
Unable to find image 'ghcr.io/psv-19/cloud-app:v1.0' locally
v1.0: Pulling from psv-19/cloud-app
90d05db799c7: Pull complete
e654dbbbb9e1: Pull complete
abaae85d1626: Pull complete
94d083cf706a: Pull complete
fbaed3f7fcbe: Pull complete
43f834d60d8a: Pull complete
a5008f4a4b25: Pull complete
de1b677d8c00: Pull complete
6a0ac1617861: Pull complete
bd5df978c10b: Download complete
Digest: sha256:fa485401797d3f326a09de828a8fa0fdddf491b382ad8bc248b2e7607ced97da
Status: Downloaded newer image for ghcr.io/psv-19/cloud-app:v1.0
675475e96004b23231c91d2f759ceafd0fc2551ba46bc106e04043c869948cf9

C:\Users\联想\docker-lab>
```


Caption: Shows pull process with each layer Pull complete, final container ID: 675475e96004

4. Experimental Data Record

Item	Content
v1.0 image digest	sha256:fa485401797d3f326a09de828a8fa0fdddf491b382ad8bc248b2e7607ced97da
v2.0 image digest	sha256:8dc7b12d1c11bdc639cfc176dc71a266dae4c9c3b08bc470e25fb79d6581faaf
cloud-lab-image:v1 size	92.7MB
Space reclaimed	818.1MB

5. Reflection

5.1 Problems Encountered and Solutions

Problem	Solution
Windows CMD does not support export command	Use CMD syntax set or directly use echo token | docker login
Docker Hub pull timeout/failure	Configure domestic image accelerators (docker.1ms.run, docker.xuanyuan.me, atomhub.openatom.cn)
Docker cannot pull images when VPN is on	Configure HTTP proxy http://127.0.0.1:7890 in Docker Desktop Settings → Resources → Proxies
Port 8888 conflict	Use docker stop <container ID> to stop old container, or use -p 8889:80
Docker Hub anonymous pull rate limit	Login to Docker Hub or use image accelerators to bypass limits

5.2 Efficiency Analysis: Why was pushing v2.0 much faster than pushing v1.0?
The reason is Docker's Layer Caching mechanism:

When pushing v1.0, all layers were uploaded for the first time, requiring a full transfer of 10 layers (approx. 92.7MB)

When pushing v2.0, only the layer generated by modifying index.html was new; the remaining 9 layers were reused from the cache, showing Layer already exists

The actual data transferred was reduced from 92.7MB to just a few KB (only the changed file layer)

This mechanism is crucial in cloud computing: each iteration only uploads the changed "fragments," greatly saving bandwidth and storage costs, while speeding up deployment.

5.3 Security Analysis: Why is PAT more secure than a GitHub password?

Comparison	GitHub Password	Personal Access Token (PAT)
Permission scope	Full account access	Limited (only write:packages)
Revocation method	Changing password affects all services	Can be revoked independently without affecting password
Expiration	Permanent (unless manually changed)	Can be set with short expiration (e.g., 7 days)
Leakage risk	Attacker gains full account control	Attacker can only access container registry

Conclusion: PAT follows the principle of least privilege. Even if leaked, it does not result in full compromise of the GitHub account, making it a security best practice in cloud-native environments.

6. Conclusion

This lab successfully completed the following tasks:
Generated and configured GitHub PAT, securely logged into GHCR using docker login
Built cloud-lab-image:v1 image, tagged and pushed to GitHub Container Registry
Modified code to build v2.0 image, observed Docker's layer caching mechanism (Layer already exists) during push
Wiped all local images to simulate a disaster scenario
Pulled image from cloud and successfully ran v1.0 service
Resolved port conflict issue and successfully deployed v2.0

This lab verified Docker's layered storage mechanism and the practicality of cloud registries. Once the application code is decoupled from the runtime environment, services can be quickly restored on any machine with Docker installed, without needing to rebuild or access source code. The use of Personal Access Tokens demonstrates security best practices in cloud-native environments.